WEB SCRAPER · V1.0.0

Da URL ao JSON.

Guia de estudo e manutenção: HTTP, HTML, extração de dados, testes e evolução do projeto.

Uma página por execução. Código pequeno, decisões explícitas.

Projeto de Arthur Volpato · github.com/volpszz/web-scraper
Material em português · Fonte editável: docs/GUIA_DO_PROJETO.md
Autenticação na v2: planejada, não implementada nesta edição.

Este documento acompanha o código deste repositório. Ele serve tanto para aprender conceitos de web com um projeto pequeno quanto para localizar o ponto correto de uma modificação. Os identificadores de código estão em inglês; as explicações, em português.

A edição PDF inclui este conteúdo e um apêndice com o código de `main.py` e `functions/scraping.py`, numerado pelas linhas originais. O Markdown continua sendo a fonte editável; o PDF é a versão para leitura e compartilhamento.

1. O que esta versão entrega

2. Preparar e executar

3. O caminho de uma requisição

4. Entender uma URL

5. HTML, DOM e JavaScript

6. Mapa do código

7. Extração de título e links

8. JSON e persistência

9. Testes e diagnóstico

10. Como modificar com segurança

11. Limitações e uso responsável

12. Exercícios e referências

1. O que esta versão entrega

Contrato da v1: receber uma URL HTTP/HTTPS, obter uma página declarada como HTML, extrair o título e os destinos dos links e salvar o resultado em JSON.

O programa:

- Rejeita entradas estruturalmente inválidas e pede outra URL.
- Faz um GET com timeout e acompanha redirecionamentos HTTP.
- Informa erros de rede e status HTTP de erro.
- Verifica o tipo do conteúdo antes de extrair título ou links.
- Extrai o primeiro `<title>`, ou retorna `None` se o elemento não existe.
- Coleta os destinos de `a[href]`, resolve links relativos, remove fragmentos e duplicatas.
- Mantém somente endereços HTTP/HTTPS que passam pela validação estrutural básica.
- Mostra os dados no terminal e salva `results/result.json`.

Ele **não visita os links coletados**. Scraping é extração; crawling é percorrer endereços. Um programa pode fazer ambos, mas esta versão só extrai de um documento por execução. Redirecionamentos podem gerar requisições adicionais, mesmo sem crawling.

O que significa estar pronto?

Pronto para este escopo educacional não significa pronto para qualquer site ou para produção. A v1 está concluída quando o caminho de entrada até o arquivo funciona, os erros previstos têm tratamento e os testes conferem essas decisões. Mais recursos só devem entrar com uma necessidade definida.

2. Preparar e executar

Na raiz do projeto, crie o ambiente virtual uma vez:

```
python -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

No Linux, você pode executar sem ativar o ambiente:

```
.venv/bin/python main.py
```

Ou ativar primeiro, usando o script do seu shell:

```
# Fish
source .venv/bin/activate.fish
python main.py
```

```
# Bash/Zsh
source .venv/bin/activate
python main.py
```

No Windows PowerShell, use `.venv\Scripts\python.exe` no lugar de `.venv/bin/python`; a ativação, se desejada, é `.venv\Scripts\Activate.ps1`.

Por que ambiente virtual?

O ambiente virtual separa as bibliotecas do projeto das do sistema. Ele não transforma Python em Fish: Fish é o shell, que interpreta comandos; Python é o interpretador que executa o programa.

`python -m pip` usa o pip associado àquele Python. Isso evita instalar uma dependência em um interpretador e executar o programa com outro.

O arquivo `requirements.txt` lista nomes de pacotes instaláveis:

```
requests
beautifulsoup4
```

Já `from bs4 import BeautifulSoup` é uma instrução Python e pertence ao arquivo `.py`. O nome distribuído pelo pip e o nome importado nem sempre coincidem.

Editor e terminal são configurações distintas

No VS Code, selecione o executável `.venv/bin/python` em **Python: Select Interpreter**. Ativar o ambiente em um terminal não garante que o Pylance esteja analisando esse mesmo ambiente.

Para diagnosticar:

```
.venv/bin/python -c "import sys, bs4; print(sys.executable); print(bs4.__file__)"
```

Se esse comando funciona mas o aviso do editor permanece, confira o interpretador e recarregue a janela com **Developer: Reload Window**. Não esconda o aviso desativando a análise, nem acrescente caminhos de bibliotecas de outro ambiente para mascarar a causa.

3. O caminho de uma requisição

Uma visão simplificada de acessar uma URL HTTPS:

```
URL informada
    ↓
Validação local da estrutura
    ↓
Resolução DNS do hostname, quando necessária
    ↓
Conexão ao destino e negociação TLS para HTTPS
    ↓
Requisição HTTP GET
    ↓
Resposta: status + cabeçalhos + corpo
```

```
    ↓
Possíveis redirecionamentos
    ↓
Verificação da resposta final → extração → JSON
```

Proxies, caches e conexões reutilizadas podem mudar os detalhes. O desenho é conceitual, não uma captura de tráfego desta execução.

DNS não é HTTP

DNS permite descobrir endereços de rede associados a um nome. Uma falha de resolução ocorre antes de receber uma resposta HTTP do servidor de destino.

Por isso, `Failed to resolve` não equivale a `404`:

- Falha de DNS: não conseguimos chegar à etapa de resposta HTTP desse destino.
- `404`: um servidor respondeu dizendo que não encontrou o recurso.

HTTPS não é uma permissão para coletar

HTTPS usa TLS para proteger a comunicação e verificar o certificado do servidor. O `requests` verifica certificados por padrão. Não desative isso com `verify=False` para fazer um erro desaparecer.

HTTPS não garante que o conteúdo seja correto, que o servidor seja confiável em todos os aspectos ou que a coleta esteja autorizada.

O método GET

GET solicita uma representação de um recurso. Neste projeto, usamos `requests.get(url, timeout=10)` e não enviamos formulários, credenciais ou alterações ao site.

Mesmo GET pode consumir recursos e registrar acessos. Não faça loops agressivos só por ser uma leitura.

Status HTTP

Faixa	Ideia geral	Exemplos
2xx	Sucesso no nível HTTP	`200 OK`
3xx	Redirecionamento ou outra condição dessa classe	`301`, `302`
4xx	A requisição não pôde ser atendida por uma condição atribuída ao cliente/acesso	`403`, `404`, `429`
5xx	Falha no lado do servidor	`500`, `503`

`requests.get()` não lança uma exceção apenas porque recebeu `404`. A chamada `response.raise_for_status()` converte status 4xx/5xx em `HTTPError`.

`HTTPError` é uma subclasse de `RequestException`. Seu `except` específico precisa vir antes do genérico para produzir uma mensagem distinta. Um `return` depois do tratamento encerra a função: o chamador recebe `None` e não continua a extração.

HTTP bem-sucedido não significa coleta correta. Um site pode responder `200` com uma página de login, um desafio contra bots ou uma tela de erro. Este scraper não detecta todas essas situações.

Cabeçalhos e corpo

O corpo está em `response.content` como bytes; `response.text` é sua representação decodificada como texto. A codificação escolhida pelo cliente importa: se o servidor declarar uma codificação errada, caracteres podem ficar incorretos. O projeto usa o comportamento padrão do `requests`, sem uma política própria de detecção.

Cabeçalhos descrevem a resposta. Exemplo:

```
Content-Type: text/html; charset=utf-8
```

A v1 faz:

```
content_type = response.headers.get("Content-Type", "")
media_type = content_type.split(";", 1)[0].strip().lower()
```

1. `.get(..., "")` evita erro se o cabeçalho estiver ausente.
2. `.split(";", 1)` separa o tipo de seus parâmetros, no máximo uma vez.
3. `[0]` escolhe o tipo.
4. `.strip().lower()` normaliza espaços externos e letras.

Somente `text/html` e `application/xhtml+xml` continuam. Ausência do cabeçalho é rejeitada: preferimos uma regra explícita a adivinhar. O cabeçalho é uma declaração do servidor, não uma prova do formato real.

XHTML é aceito, mas passa pelo mesmo `html.parser`; não oferecemos parsing XML estrito ou tratamento completo de namespaces.

Timeout: um detalhe importante

`timeout=10` não é um cronômetro de dez segundos para o programa inteiro. O valor controla esperas de conexão e leitura do `requests`; em particular, a espera de leitura se relaciona à chegada de dados, não ao tempo total de download. DNS, redirecionamentos e um servidor que envia dados lentamente podem fazer a execução durar mais.

4. Entender uma URL

Considere este exemplo didático:

```
https://example.com:8443/blog/artigo?page=2#comentarios
```

Parte	Valor	Papel
Esquema	`https`	Como acessar o recurso
Hostname	`example.com`	Nome do servidor
Porta	`8443`	Porta explícita de conexão
Caminho	`/blog/artigo`	Identificação do recurso no servidor
Query	`page=2`	Parâmetros cujo significado depende da aplicação
Fragmento	`comentarios`	Referência dentro do documento; não é enviado como parte do alvo HTTP

`urlsplit()` separa componentes, mas não substitui uma política de validação. Em `is_valid_url()` acrescentamos estas regras:

- Exigir esquema HTTP/HTTPS e hostname.
- Rejeitar espaços e caracteres de controle.
- Rejeitar usuário/senha embutidos na URL.
- Acessar `parsed.port` para detectar porta inválida ou fora da faixa; rejeitar também zero.
- Verificar se o hostname pode ser codificado como IDNA, capturando alguns nomes inválidos.
- Capturar `ValueError` e `UnicodeError`, retornando `False` em vez de derrubar o programa.

A entrada interativa recebe `.strip()`: espaços acidentais nas extremidades são removidos antes da validação. Espaços internos continuam inválidos; quando necessários em um caminho, devem estar corretamente codificados, como `%20`.

O que a validação não garante: existência do domínio, autorização, disponibilidade, sintaxe exaustiva de todos os casos de hostname ou segurança contra SSRF. Ela também aceita localhost e IPs privados para uso local e testes.

5. HTML, DOM e JavaScript

HTML representa a estrutura de um documento:

```
<html>
  <head><title>Meu projeto</title></head>
  <body>
    <h1>Bem-vindo</h1>
    <a href="/sobre">Sobre</a>
  </body>
</html>
```

- `<title>` costuma fornecer o título da aba, não o título visual principal.
- `<h1>` é um título do conteúdo.
- `<a>` representa um link; `href` contém o destino.
- CSS altera a apresentação; não é necessário renderizá-lo para ler esses dados.

No navegador, o HTML é interpretado em uma árvore de objetos: o DOM. JavaScript pode modificar essa árvore, criar elementos e buscar dados de APIs.

O `requests` recebe a resposta HTTP, **não o DOM final após JavaScript**. Beautiful Soup cria uma árvore de parsing a partir desse HTML, mas não executa scripts nem simula um navegador.

Um site feito com React ou Next.js não exige automaticamente Selenium: se os dados estiverem no HTML entregue pelo servidor, nossa abordagem funciona. Se aparecerem apenas depois de JavaScript, investigue o HTML recebido e a aba Network do navegador. Uma API documentada pode ser mais adequada; um navegador automatizado só deve entrar quando realmente necessário.

6. Mapa do código

```
main.py                     entrada do programa e código de saída
functions/
  greetings.py              apresentação no terminal
  scraping.py               validação, HTTP, extração, apresentação e JSON
requirements.txt            dependências externas
README.md                   instalação e consulta rápida
.gitignore                  exclusões do Git, incluindo results/
tests/
  test_scraping.py           sucesso HTTP e falhas simuladas
  test_validation.py         entradas válidas e inválidas
  test_extraction.py         título, links, filtros e duplicatas
  test_storage.py            JSON, Unicode, diretórios e erro de escrita
  test_flow.py               fluxo integrado, servidor local e CLI real
docs/
  GUIA_DO_PROJETO.md         este documento
results/
  result.json               saída gerada; não versionada
```

Fluxo efetivo

```
main.py
  ├── greetings.startup()
  └── initial_scrap()
        ├── input() → is_valid_url() → repetir se inválida
        ├── fetch_page() → encerrar em falha
        ├── show_response()
        ├── conferir Content-Type → encerrar se não HTML
        ├── extract_title(response.text)
        ├── extract_links(response.text, response.url)
        ├── montar dicionário do resultado
        └── save_result() → informar caminho ou falha
```

Contratos das funções

Função	Entrada	Saída / comportamento
`is_valid_url(url)`	String	Booleano; não faz rede
`fetch_page(url)`	URL válida	`requests.Response` ou `None`; informa erros esperados
`extract_title(html)`	HTML em texto	String ou `None`
`extract_links(html, base_url)`	HTML e URL de referência	Lista de URLs; não faz rede
`show_response(response)`	Resposta HTTP	Imprime diagnóstico; não extrai os dados
`save_result(result, output_path)`	Dicionário serializável e caminho	Escreve JSON e retorna `Path`; erros de disco chegam ao chamador
`initial_scrap(output_path)`	Caminho opcional	Coordena entrada e saída; retorna `True` apenas após salvar

`main.py` transforma sucesso em código de saída `0`; falha ou cancelamento, em `1`. Isso permite que outro programa detecte falhas sem interpretar frases impressas.

Por que manter simples?

Não foi necessário adicionar classes, banco de dados, framework ou async. As funções têm contratos pequenos. O título e os links fazem parsing separadamente: é um pouco de trabalho duplicado, mas a separação é clara para esta escala. Se o volume justificar, você pode compartilhar uma árvore de parsing depois de medir e preservar os testes.

Mantivemos os nomes e as funções que você já vinha construindo. Para comparar com a base inicial versionada, use `git show 5e7c24a:functions/scraping.py`; esse commit não inclui todas as alterações feitas durante as etapas de aprendizado.

7. Extração de título e links

Título

`BeautifulSoup(html, "html.parser")` interpreta o documento usando o parser HTML da biblioteca padrão. `find("title")` devolve o primeiro elemento correspondente ou `None`.

Precisamos testar a ausência antes de chamar `get_text(strip=True)`. Uma página sem título continua sendo um resultado válido. Um elemento `<title></title>` existe e produz string vazia, diferente de ausência.

O parser também interpreta entidades HTML: `&` vira `&` no texto extraído.

Links: do elemento à URL

Exemplo didático:

```
<a href="artigo#resumo">Ler artigo</a>
```

`find_all("a", href=True)` seleciona âncoras com o atributo. `element["href"]` lê `artigo#resumo`; não lê o texto `Ler artigo`.

O algoritmo:

1. Limpa espaços nas extremidades do atributo.
2. Ignora destino vazio e links começando com `#`.
3. Ignora espaços internos e caracteres de controle.
4. Usa `urljoin()` para obter uma URL absoluta.
5. Usa `urldefrag()` para remover o fragmento.
6. Reutiliza `is_valid_url()` para filtrar destinos HTTP/HTTPS válidos segundo nossa política.
7. Usa um conjunto para detectar URLs repetidas e uma lista para manter a ordem.

Um `href` malformado, como `http://[broken`, pode fazer `urljoin()` lançar `ValueError`. Capturamos esse erro por link: um elemento ruim não deve impedir que os demais sejam coletados.

Referências relativas

URL da página	href	Resultado
`https://example.com/blog/`	`post`	`https://example.com/blog/post`
`https://example.com/blog/`	`/sobre`	`https://example.com/sobre`
`https://example.com/blog/index.html`	`post`	`https://example.com/blog/post`
`https://example.com/blog/`	`../sobre`	`https://example.com/sobre`
`https://example.com/blog/`	`//outro.example/path`	`https://outro.example/path`

A barra final e a barra inicial do destino importam. `urljoin()` aplica regras de resolução; não é apenas concatenação de strings.

Usamos `response.url` como referência porque um redirecionamento pode alterar o endereço do documento. O endereço digitado fica em `source_url`; o final, em `url`.

Fragmentos, queries e duplicatas

`/sobre#equipe` e `/sobre#contato` viram a mesma URL sem fragmento. Porém, `/itens?page=1` e `/itens?page=2` continuam distintos, porque a query pode mudar o recurso.

O conjunto `seen` usa a string final como chave. Isso não é canonicalização completa: duas URLs que um servidor considera equivalentes podem permanecer distintas por barra final, ordem da query ou outras variações.

`mailto:`, `tel:` e `javascript:` não são destinos de páginas HTTP/HTTPS, portanto ficam de fora. Links para outros domínios são aceitos, mas **não acessados**. Um link para PDF pode entrar na lista: coletar um destino não significa que o conteúdo desse destino seja HTML.

Mostrar a lista não é navegar

```
links = extract_links(response.text, response.url)
print("Links found:", len(links))
for link in links:
    print("-", link)
```

A chamada devolve uma lista. `len()` conta itens já filtrados e deduplicados, não todos os elementos `<a>`. O `for` só imprime as strings; não há `requests.get()` dentro dele.

8. JSON e persistência

O resultado é um dicionário Python convertido em JSON. Exemplo **didático**, não uma resposta capturada:

```
{
  "source_url": "https://example.com/entrada",
  "url": "https://example.com/blog/",
  "status_code": 200,
  "content_type": "text/html; charset=utf-8",
  "fetched_at": "2026-01-01T12:00:00+00:00",
  "title": "Meu blog",
  "links": ["https://example.com/sobre"],
  "link_count": 1
}
```

- `source_url`: entrada após retirar espaços externos.
- `url`: destino final da resposta.
- `fetched_at`: instante de montagem do resultado em UTC, não a data de publicação nem o instante exato do primeiro byte.
- `title`: string, ou `null` em JSON quando Python retorna `None`.
- `links`: lista ordenada.
- `link_count`: precisa ser sempre igual a `len(links)`.

Por que JSON?

É legível, representa naturalmente dicionários/listas e permite que outro programa consuma a saída. Não é um banco de dados e não fornece histórico, consultas ou controle de concorrência.

Como `save_result()` trabalha

1. Converte o caminho para `Path`.
2. Serializa com `json.dumps(..., ensure_ascii=False, indent=2)`.
3. Cria a pasta de saída se necessário, inclusive seus diretórios pais.

4. Escreve com `encoding="utf-8"`.
5. Devolve o caminho; o coordenador só anuncia sucesso depois disso.

`ensure_ascii=False` mantém acentos legíveis; `indent=2` formata o arquivo. O caminho padrão deriva de `__file__`, não da pasta atual do terminal, para manter o resultado dentro deste projeto.

Política de sobrescrita: cada sucesso substitui `results/result.json`. Falhas anteriores à escrita não tocam no resultado antigo. Uma falha durante a escrita pode deixar arquivo parcial: a v1 não usa substituição atômica nem backup. Se os dados forem importantes, evolua esse ponto antes de depender dela.

`results/` está no `.gitignore`, porque dados coletados podem mudar e conter informações que você não deseja publicar. Isso não é controle de acesso nem criptografia; evite coletar ou compartilhar dados sensíveis.

Para mudar o destino ao usar a função em outro código, passe um caminho a `initial_scrap(output_path)` ou a `save_result(result, output_path)`. A interface de terminal da v1 não tem argumento `--output`.

9. Testes e diagnóstico

Execute na raiz:

```
.venv/bin/python -m unittest discover -s tests -v
```

Três camadas de confiança

Testes locais de lógica: passam strings HTML e URLs diretamente às funções. São rápidos e não dependem da internet. Confira ausência de título, links relativos, duplicatas, esquemas excluídos e entradas malformadas.

Falhas HTTP simuladas: os testes de `fetch_page()` controlam a resposta de `requests.get()` para verificar status e exceções sem depender de DNS ou de um servidor externo. Isso prova nosso tratamento, não o funcionamento da rede externa.

Integração real em loopback: `test_flow.py` sobe um `ThreadingHTTPServer` em `127.0.0.1`, porta escolhida pelo sistema, com rotas controladas. Verifica redirecionamento, HTML UTF-8, JSON, cabeçalho ausente, página sem dados e erro 404. O servidor é encerrado depois dos testes.

Há também um subprocesso real do CLI em uma cópia temporária isolada do projeto, para conferir execução, caminho padrão e códigos de saída sem sobrescrever seu resultado de uso manual. Os temporários usam a configuração de diretório temporário do ambiente, incluindo `TMPDIR` quando definido.

Um teste de integração verifica o JSON lendo o arquivo de volta. Uma mensagem `Saved to:` sozinha não provaria que os dados foram escritos corretamente.

Test-first em uma mudança

1. Defina o comportamento esperado.

2. Escreva o menor teste que demonstre a ausência ou o defeito.
3. Execute e confirme que falha pelo motivo esperado.
4. Faça a menor alteração de produção.
5. Execute o teste e depois a suíte inteira.
6. Só então reorganize o código, mantendo tudo verde.

Testes não garantem ausência de bugs. Um caso que nunca foi representado pode revelar uma falha nova. Ao encontrá-la, primeiro adicione uma reprodução.

Diagnóstico prático

Sintoma	Investigue primeiro
`ModuleNotFoundError: bs4`	`sys.executable`, instalação no `.venv` e `requirements.txt`
Só o Pylance acusa o import	Interpretador do editor e recarregamento da janela
`Failed to resolve`	Nome digitado, DNS e conectividade; não tratar como 404
`403`	Permissão e regras do serviço; não tentar contornar
`429`	Limite de requisições; interromper e respeitar instruções do servidor
`200`, mas dados errados	Corpo recebido, login/desafio e regras de extração
Página visível tem links, HTML recebido não	Possível conteúdo criado por JavaScript
Acentos errados	Cabeçalho de codificação e bytes recebidos
`Error saving JSON`	Caminho, permissões, espaço em disco e arquivo/diretório no destino
JSON parece de outra execução	Verifique se houve sucesso e compare `fetched_at` e `source_url`

10. Como modificar com segurança

Exemplo: adicionar a descrição da página

Um incremento útil seria extrair `<meta name="description" content="...">`.

Não é preciso alterar a função HTTP. O caminho é:

1. Em `test_extraction.py`, crie casos com e sem descrição, incluindo `content` ausente.
2. Crie `extract_description(html)` em `functions/scraping.py`.
3. Faça a função retornar uma string ou `None`, sem imprimir ou fazer rede.

4. Em `initial_scrap()`, chame-a somente depois do filtro HTML.
5. Acrescente `description` ao dicionário do resultado.
6. Atualize o teste integrado e a tabela do JSON no README.

Antes de implementar, decida como tratar várias descrições, string vazia e atributos com variações de caixa. O teste deve registrar a decisão, não apenas um exemplo que deu certo.

Onde alterar cada responsabilidade

Necessidade	Lugar principal	O que testar
Novo dado do HTML	Função de extração + montagem do resultado	Elemento presente, ausente e incompleto
Outro caminho/formato de saída	`save_result()` ou nova função de armazenamento	Conteúdo, codificação e falhas de disco
Outra política de links	`extract_links()`	Ordem, filtros, duplicatas e casos relativos
Timeout ou política HTTP	`fetch_page()`	Exceções e integração com servidor controlado
Nova regra de entrada	`is_valid_url()`	Entradas aceitas/rejeitadas sem chamadas de rede
Flags no terminal	Camada de entrada, eventualmente `argparse`	CLI real e códigos de saída

Se quiser coletar várias páginas

Não coloque simplesmente um GET dentro do `for` que imprime links. Isso muda o escopo e introduz ciclos, repetição, crescimento de fila, carga no site e riscos de acessar destinos inesperados.

Defina antes: domínios permitidos, máximo de páginas, profundidade, intervalo, política de `robots.txt`, timeouts, retries limitados, limites de corpo e registro de falhas. Mantenha um conjunto de visitados e revalide cada destino, inclusive após redirecionamentos.

11. Limitações e uso responsável

A v1 é uma ferramenta local de aprendizado. Limites intencionais:

- Não executa JavaScript, não autentica, não mantém uma sessão de login e não burla proteções.
- Não interpreta `<base href>`; usa a URL final do documento como base.
- Não procura links dentro de documentos de iframe, Shadow DOM, CSS ou scripts.
- Não valida se os destinos coletados existem.

- Não limita o tamanho da resposta nem implementa prazo total, fila, concorrência ou retry próprio.
- Não possui escrita atômica, histórico, banco de dados ou coordenação entre execuções simultâneas.
- Não fixa versões das dependências nem oferece lockfile; uma instalação futura pode trazer versões diferentes. Se precisar de reprodutibilidade, defina uma política de versões e teste-a.
- O preview de 500 caracteres é apenas de apresentação: o corpo completo já foi baixado e pode conter dados privados.

Use sites seus ou autorizados, leia termos e respeite restrições de acesso e coleta. `robots.txt` comunica regras para crawlers; não é autorização legal nem mecanismo de autenticação. Esta versão não o consulta automaticamente. Não amplie a coleta sem uma política adequada.

SSRF: por que não publicar como uma API genérica?

Se terceiros puderem fornecer URLs a um serviço seu, poderão tentar fazer seu servidor acessar endereços internos, localhost ou serviços de metadados. Isso é uma classe de risco chamada SSRF.

Nossa validação básica aceita endereços locais de propósito e **não protege contra isso**. Um serviço exposto precisa de política de destinos, resolução e verificação de IPs, revalidação de redirecionamentos, restrições de rede e limites de recursos. Um `startswith("https://")` não resolve esse problema.

12. Exercícios e referências

Exercícios progressivos

1. **Descrição:** extraia a meta description. Critério: ausência e atributo incompleto não quebram o programa.
2. **Texto dos links:** devolva objetos com URL e texto. Decida qual texto manter quando duas âncoras levam ao mesmo destino.
3. **Suporte a `<base href>`:** use o primeiro elemento válido como referência. Teste base relativa, absoluta e inválida.
4. **Histórico:** crie um arquivo por execução. Critério: duas execuções não sobrescrevem uma à outra; documente o nome e o fuso do timestamp.
5. **Escrita atômica:** escreva em arquivo temporário no mesmo diretório e substitua o destino ao concluir. Simule falha e confira preservação do arquivo anterior e limpeza do temporário.
6. **CLI com `argparse`:** aceite URL e `--output` como argumentos, preservando uma política clara para modo interativo e erros.
7. **Um domínio só:** filtre links pelo hostname da URL final. Teste subdomínios e nomes parecidos para não usar comparação insegura por substring.

8. **Crawler limitado:** só depois dos anteriores, implemente uma fila com limites explícitos e testes em servidor local. Não use a internet como ambiente de carga.

Em todos: formule o contrato, observe o teste falhar, implemente, execute a suíte e atualize a documentação.

Perguntas para conferir entendimento

- Por que DNS falhando não produz status 404?
- Por que um status 200 pode não conter o conteúdo que você queria?
- Por que `response is not None` é mais adequado que `if response` aqui? Objetos `Response` do requests podem ser falsos em status de erro, embora existam.
- Por que a validação do tipo de conteúdo vem antes das duas extrações?
- Qual a diferença entre `/post`, `post`, `?page=2` e `#section` em um `href`?
- Por que resolver links usando a URL final, e não sempre a digitada?
- O que `None` vira em JSON? E qual a diferença entre título ausente e título vazio?
- Qual teste precisaria mudar se você decidisse preservar fragmentos?
- Como saber que o arquivo salvo corresponde à última tentativa bem-sucedida?
- Quais limites faltam antes de transformar isto em um serviço público?

Referências para aprofundar

- MDN — Visão geral do HTTP
- MDN — URLs
- Requests — Quickstart, especialmente erros, redirecionamentos, conteúdo e timeouts.
- Beautiful Soup — documentação
- Python — urllib.parse, incluindo os avisos de segurança de parsing e `urljoin`.
- Python — json
- Python — unittest
- Python — venv

Sugestão de estudo: execute primeiro, leia `main.py`, siga `initial_scrap()` e consulte uma função por vez. Depois use os testes como exemplos executáveis dos contratos. Não comece pela adição de um framework: comece pelo dado que quer extrair e pelo comportamento que consegue demonstrar.

Reproduzir o PDF

As ferramentas de documentação são opcionais e não entram em `requirements.txt` do scraper:

```
python -m venv .venv-docs
.venv-docs/bin/python -m pip install -r requirements-docs.txt
.venv-docs/bin/python -m playwright install chromium
.venv-docs/bin/python scripts/build_guide_pdf.py
.venv-docs/bin/python -m unittest discover -s scripts -p test_build_guide_pdf.py -v
```

O gerador usa Chromium do Playwright ou detecta um navegador Chromium/Chrome/Brave instalado. Você pode indicar outro executável com `--browser /caminho/do/navegador` ou a variável `PDF_BROWSER`. Ele usa fontes do sistema, preferindo DejaVu Sans e DejaVu Sans Mono com alternativas padrão. No Windows, substitua `.venv-docs/bin/python` por `.venv-docs\Scripts\python.exe`.

Saídas: `docs/GUIA_DO_PROJETO.pdf` e `docs/GUIA_DO_PROJETO.manifest.json`. O manifesto registra SHA-256 dos arquivos de entrada e do PDF, além da quantidade de páginas. Depois de alterar os fontes explicados, o guia ou o gerador, gere novamente. O teste da documentação renderiza um PDF pequeno e confere texto, tabela, código e destinos de navegação; a revisão visual continua necessária para o documento final.

O plano futuro de autenticação fica separado em `docs/ROADMAP_V2.md`. A v1.0.0 documentada aqui não implementa login.

Código de referência: main.py

Números são as linhas originais. Uma célula pode ocupar várias linhas visuais; a numeração não é reiniciada nas continuações.

```
 1 import functions.greetings as greetings
 2 import functions.scraping as scraping
 3
 4 if __name__ == "__main__":
 5     greetings.startup()
 6     try:
 7         success = scraping.initial_scrap()
 8     except KeyboardInterrupt:
 9         print("\nCancelled.")
10         success = False
11     raise SystemExit(0 if success else 1)
```

Código de referência: functions/scraping.py

Números são as linhas originais. Uma célula pode ocupar várias linhas visuais; a numeração não é reiniciada nas continuações.

```
import json
from datetime import datetime, timezone
from pathlib import Path
from urllib.parse import urldefrag, urljoin, urlsplit

import requests
from bs4 import BeautifulSoup


DEFAULT_OUTPUT_PATH = Path(__file__).resolve().parent.parent / "results" / "result.json"


def save_result(result, output_path=DEFAULT_OUTPUT_PATH):
    """Save the latest result as readable UTF-8 JSON, replacing an existing file."""
    path = Path(output_path)
    content = json.dumps(result, ensure_ascii=False, indent=2) + "\n"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content, encoding="utf-8")
    return path


def is_valid_url(url):
    """Validate URL structure, without checking DNS or reaching the server."""
    if not url or any(char.isspace() or ord(char) < 32 or ord(char) == 127 for char in url):
        return False

    try:
        parsed = urlsplit(url)
        if parsed.scheme not in ("http", "https") or not parsed.hostname:
            return False
        if parsed.username is not None or parsed.password is not None:
            return False
        if parsed.port is not None and parsed.port == 0:
            return False
        parsed.hostname.encode("idna")
    except (ValueError, UnicodeError):
        return False

    return True


def extract_title(html):
    """Return the first document title, or None if its element is missing."""
    soup = BeautifulSoup(html, "html.parser")
```

```
    title_element = soup.find("title")

    if title_element is None:
        return None

    return title_element.get_text(strip=True)


def extract_links(html, base_url):
    """Return unique HTTP/HTTPS anchor destinations in document order."""
    soup = BeautifulSoup(html, "html.parser")
    links = []
    seen = set()

    for element in soup.find_all("a", href=True):
        href = element["href"].strip()

        if not href or href.startswith("#"):
            continue

        if any(char.isspace() or ord(char) < 32 or ord(char) == 127 for char in href):
            continue

        try:
            absolute_url = urljoin(base_url, href)
            absolute_url, _ = urldefrag(absolute_url)
        except ValueError:
            continue

        if not is_valid_url(absolute_url):
            continue

        if absolute_url in seen:
            continue

        seen.add(absolute_url)
        links.append(absolute_url)

    return links


def fetch_page(url):
    """Fetch a page, following redirects; return None for request/HTTP errors."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return response
    except requests.exceptions.HTTPError as error:
        print(f"Erro HTTP ao acessar a página: {error}")
        return None
    except requests.exceptions.RequestException as error:
```

```
        print(f"Erro ao acessar a página: {error}")
        return None


def show_response(response):
    print("Status:", response.status_code)
    print("Content type:", response.headers.get("Content-Type"))
    print("Body preview:")
    print(response.text[:500])


def initial_scrap(output_path=DEFAULT_OUTPUT_PATH):
    """Run one interactive scrape; return True only after saving its result."""
    while True:
        try:
            url = input("Insert the target url: ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nCancelled.")
            return False

        if is_valid_url(url):
            break

        print("Error: enter a valid HTTP/HTTPS URL with a host, no spaces or credentials, and a valid port.")

    response = fetch_page(url)

    if response is None:
        return False

    show_response(response)

    content_type = response.headers.get("Content-Type", "")
    media_type = content_type.split(";", 1)[0].strip().lower()

    if media_type not in ("text/html", "application/xhtml+xml"):
        print("Error: response is not an HTML document.")
        return False

    title = extract_title(response.text)
    links = extract_links(response.text, response.url)

    print("Page title:", title if title is not None else "not found")
    print("Links found:", len(links))
    for link in links:
        print("-", link)

    result = {
        "source_url": url,
        "url": response.url,
```

```
        "status_code": response.status_code,
        "content_type": content_type,
        "fetched_at": datetime.now(timezone.utc).isoformat(),
        "title": title,
        "links": links,
        "link_count": len(links),
    }

    try:
        path = save_result(result, output_path)
    except OSError as error:
        print(f"Error saving JSON: {error}")
        return False

    print("Saved to:", path)
    return True
```